

# JAVA INSTITUTE FOR ADVANCED TECHNOLOGY

Department of Examinations

Business Component Development I (JIAT/BCD I)

## E-Commerce Platform Modernization Project

(TechMart Online)

|               |                                                    |
|---------------|----------------------------------------------------|
| Student Name: | Semini Wijesinghe                                  |
| NIC:          | [Student's NIC Number] (Placeholder: 200384601938) |
| Branch:       | Colombo                                            |
| Date:         | July 5, 2026                                       |
| Submission:   | Final Coursework Portfolio                         |

Technical Configurations & Concurrency Analysis Report

## 1. Executive Summary & Multi-Module Architecture

This technical document outlines the configuration profile and concurrency architecture of the modernized TechMart Online E-Commerce enterprise application. The platform has been refactored from a monolithic structure into a standard Multi-Module Enterprise Archive (EAR) architecture, facilitating improved scalability, component isolation, and thread management. The modular layout is organized as follows:

- **techmart-ejb:** Contains the core business logic components, including stateless, stateful, and singleton Enterprise JavaBeans (EJBs), JPA database models, JMS Message-Driven Beans (MDBs), and transactional boundaries.
- **techmart-war:** Manages the presentation layer, containing the web controller servlets, the browser user interface (HTML/CSS), and web container descriptors.
- **techmart-ear:** The main Enterprise Archive that packages the EJB and WAR modules along with the necessary database drivers (such as the MySQL Connector) into a single deployable archive.

## 2. Payara Server Thread Pool Configurations

The application is hosted on Payara Server 6. System resource profiles and thread bounds are defined in the server configuration file (domain.xml) under the active server-config element. These thread pools prevent resources from being exhausted during high-load periods by placing strict boundaries on concurrent executors. The configurations extracted from the Payara application server are outlined in the table below:

| Thread Pool Name  | Min Size       | Max Size         | Queue Size        | Application Context / Purpose                                                                 |
|-------------------|----------------|------------------|-------------------|-----------------------------------------------------------------------------------------------|
| admin-thread-pool | 1              | 15               | 256               | Dedicated to the Payara Admin GUI console and REST administrative command operations.         |
| http-thread-pool  | 2<br>(Default) | 200<br>(Default) | 4096<br>(Default) | Handles incoming client HTTP requests, servicing servlet routing and REST web UI traffic.     |
| thread-pool-1     | 2<br>(Default) | 200              | Unlimited         | Assigned to the IIOP ORB (Object Request Broker) service for remote EJB client communication. |

### 3. Database Configurations & JPA Persistence Properties

Database connections are managed via Java Transaction API (JTA) through a data source configured inside the EJB module. The properties mapped in the persistence.xml configuration file located in the EJB module (techmart-ejb/src/main/resources/META-INF/persistence.xml) control transaction boundaries and persistence behavior. The XML descriptor defines the persistence unit TechMartPU, linking it with the application-scoped data source java:app/jdbc/TechMartDS. Below is a clean structured representation of the XML configuration:

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="3.0" xmlns="https://jakarta.ee/xml/ns/persistence">
  <persistence-unit name="TechMartPU" transaction-type="JTA">
    <jta-data-source>java:app/jdbc/TechMartDS</jta-data-source>
    <properties>
      <property name="jakarta.persistence.schema-generation.database.action" value="
drop-and-create"/>
      <property name="eclipselink.ddl-generation" value="drop-and-create-tables"/>
      <property name="hibernate.dialect" value="org.hibernate.dialect.MySQLDialect"/
>
      <property name="hibernate.show_sql" value="true"/>
      <property name="hibernate.format_sql" value="true"/>
      <property name="hibernate.connection.release_mode" value="auto"/>
    </properties>
  </persistence-unit>
</persistence>
```

#### Key Persistence Properties & Descriptions:

| Property Name                                         | Configured Value                   | Technical Functionality                                                                        |
|-------------------------------------------------------|------------------------------------|------------------------------------------------------------------------------------------------|
| jta-data-source                                       | java:app/jdbc/TechMartDS           | Binds the persistence unit to the application server's JTA-managed connection pool.            |
| jakarta.persistence.schema-generation.database.action | drop-and-create                    | Automatically recreates the relational schema upon application deployment.                     |
| hibernate.dialect                                     | org.hibernate.dialect.MySQLDialect | Instructs Hibernate ORM to compile queries into MySQL-compatible SQL syntax.                   |
| hibernate.connection.release_mode                     | auto                               | Optimizes connection pool retention by automatically releasing connections after transactions. |

## 4. EJB Container Concurrency Architecture

The core processing layer relies on several Enterprise JavaBeans (EJBs) configured with specialized concurrency boundaries, transaction managers, and lifecycle interceptors to support thread-safe operations in high-load scenarios. The table below details these EJB concurrency profiles, outlining lock semantics, activation config properties, worker thread behavior, and transactional contexts:

| EJB Component                                                      | Concurrency Management              | Configuration Details & Thread Safety                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|--------------------------------------------------------------------|-------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>PlatformMetricsRegistry</b><br><i>(Startup Singleton)</i>       | Container-Managed Concurrency (CMC) | <ul style="list-style-type: none"><li>• <b>Annotations:</b> @Singleton, @Startup, @ConcurrencyManagement.</li><li>• <b>Locking Semantics:</b> Uses fine-grained method-level locking.<ul style="list-style-type: none"><li>- @Lock(LockType.WRITE) enforces exclusive locks for metric mutations (e.g. log updates, session changes).</li><li>- @Lock(LockType.READ) allows multi-threaded read access to metrics.</li></ul></li><li>• <b>Transaction:</b> TransactionAttributeType.NOT_SUPPORTED prevents database locks from blocking metrics registry.</li></ul> |
| <b>NotificationService</b><br><i>(Async Worker)</i>                | Asynchronous Thread Pool Execution  | <ul style="list-style-type: none"><li>• <b>Annotations:</b> @Stateless, @Asynchronous.</li><li>• <b>Concurrency Setup:</b> sendNotificationAsync spawns non-blocking worker threads. The method immediately returns a Future&lt;Boolean&gt; wrapping an AsyncResult.</li><li>• <b>Simulated Latency:</b> Integrates a Thread.sleep(1500) block to replicate SMTP network round-trip delay without blocking calling threads.</li></ul>                                                                                                                               |
| <b>OrderProcessorMDB</b><br><i>(JMS Queue Listener)</i>            | Message-Driven Pool Multi-threading | <ul style="list-style-type: none"><li>• <b>Annotations:</b> @MessageDriven, implementing MessageListener.</li><li>• <b>JMS Destination:</b> Bound to java:app/jms/OrderQueue via @JMSTDestinationDefinition.</li><li>• <b>Activation Properties:</b><ul style="list-style-type: none"><li>- destinationLookup: java:app/jms/OrderQueue</li><li>- destinationType: jakarta.jms.Queue</li><li>- acknowledgeMode: Auto-acknowledge</li></ul></li><li>• <b>Concurrency:</b> Concurrently processes messages via Payara's MDB thread pool.</li></ul>                     |
| <b>PerformanceInterceptor</b><br><i>(AroundInvoke Interceptor)</i> | Around-Invoke Call Interception     | <ul style="list-style-type: none"><li>• <b>Lifecycle Hook:</b> @AroundInvoke intercepts EJB business methods.</li><li>• <b>Metrics Tracking:</b> Measures execution time and maps the target class to its specific EJB bean type (Stateless, Stateful, Singleton, or MessageDriven).</li><li>• <b>Thread Safety:</b> Thread-safe wrapper that logs durations directly into the PlatformMetricsRegistry.</li></ul>                                                                                                                                                   |